

День 11 • 🎯 Знакомство с LangSmith, менеджмент состояний в LangGraph, контекст и иерархия

📋 Обсуждаемые темы:

- 📊 LangSmith: трейсинг и отладка
- 📁 Состояние в LangGraph: лучшие практики
- 🧠 Контекст в мультиагентных системах
- 🏗️ Иерархия и координация агентов



LangSmith: что это и зачем

LangSmith — сервис «наблюдаемости» для цепочек и агентов: видно, что вызывалось, что передавалось, сколько это заняло и сколько стоило.

Зачем: трейсинг вызовов к моделям и инструментам, пошаговая отладка (промпы, ответы, состояние), оценка качества пайплайнов и мониторинг в продакшене.

LangSmith: как подключить

Регистрация на `smith.langchain.com`, получение API -ключа.

В настройках окружения включается трейсинг и подставляется ключ; при желании задаётся имя проекта, куда складывать трейсы. После этого вызовы из приложения автоматически попадают в `LangSmith`.

Дашборд — смотреть трейсы, цепочки, затраты токенов и времени.

Отладка — клик по шагу и просмотр промптов, ответов и состояния.

Оценка качества — прогон датасетов и метрики. В продакшене — мониторинг задержек и ошибок. При необходимости можно включать трейсинг только для нужных вызовов.

Состояние в LangGraph: зачем оно нужно

Состояние — как общая доска для команды: все агенты смотрят на неё и дописывают своё. Один общий «источник правды».

Что на доске: история переписки, текущая задача, промежуточные результаты, кто сейчас «ведущий», статус работы.



Из чего состоит состояние

-  Сообщения — история диалога (кто что сказал).
-  Текущая задача и промежуточные результаты.
-  Метаданные — статус (планируем, делаем, проверяем, готово) и кто сейчас ведёт процесс.



Как правильно «дописывать» состояние

Аналогия: если несколько человек пишут в один общий список, нельзя просто подменять весь список последней записью — иначе предыдущие пропадут. Нужно явно договориться: «добавляем к существующему», а не «заменяем».

Тезис: для всего, что накапливается (сообщения, результаты шагов), задавайте правило слияния — иначе система по умолчанию подставит только последнее значение и потеряет остальное.



Сохранение и восстановление

Аналогия: как автосохранение в игре — после каждого шага делается снимок. Упало — продолжаем с последней точки. Можно «остановиться для проверки человеком» — тогда у диалога есть свой идентификатор сессии, чтобы потом возобновить именно его.

Тезисы: снимки на каждом шаге, продолжение после сбоя, пауза для человека с возможностью вернуться к тому же разговору.



Контекст в мультиагентных системах

Контекст — всё, что агенты учитывают при решении: что уже было сказано, какая цель, в какой предметной области работаем.

Типы: история разговора, формулировка задачи, знания о домене. Задача — передавать между агентами только нужное, отфильтровать лишнее и собрать данные из разных источников.

-  У модели ограничен «объём памяти» — нельзя запихнуть бесконечно много текста.
-  На каждом шаге нужна не вся история, а релевантная часть.
-  Чем больше контекста — тем дольше и дороже ответ.

Брать из истории только то, что относится к текущей задаче, и укладываться в лимит «памяти» модели: идти с конца (последнее важнее), добавлять сообщения по релевантности и останавливаться, когда лимит достигнут.

- **Скользящее окно** — хранить только последние N сообщений, как в переписке: видно только недавнее.
- **Сжатие в саммари** — старую историю не отдавать целиком, а кратко пересказать ключевое.
- **Отбор по смыслу** — оставлять только то, что «по теме» текущего вопроса (по близости смысла, а не просто по порядку).



Оставлять в контексте не «последние пять реплик», а те сообщения, которые по смыслу ближе всего к текущему вопросу — как поиск по смыслу внутри истории разговора.

Супервизор: один «главный» агент раздаёт задачи остальным и принимает решения — как менеджер в команде.

Дерево: несколько уровней — менеджеры координируют группы агентов, те — своих подчинённых. Масштабируется на большие системы.

- **Командно-контрольный (менеджер)** — решения сверху вниз, чёткий план. Подходит для стандартизированных процессов.
- **Коллаборативный (рой)** — решения вместе, коллективный интеллект, гибкость при изменениях.

Состояние: хранить только необходимое; для всего, что накапливается, задавать правило «добавлять, а не заменять»; данные должны уметь сохраняться в снимки для восстановления.

Контекст: следить за размером, использовать скользящее окно и отбор по смыслу.

Иерархия: начинать с простой структуры, чётко распределять роли.



Выводы и рекомендации

Принципы: состояние — единая «доска» для всех; контекст — только релевантная информация; иерархия — чёткие роли и зоны ответственности.

Инструменты: визуальная отладка графа, LangSmith для трейсинга и оценки качества, тесты на логику графа.

Вы получили новые инструменты — но опыт начинается с практики... 

 **Исследуйте** — пробуйте код, экспериментируйте, ошибайтесь

 **Углубляйтесь** — каждая тема открывает новые возможности

 **Создавайте** — даже маленький проект лучше теории

Это невозможно, пока вы это не сделаете!

Успехов! 